

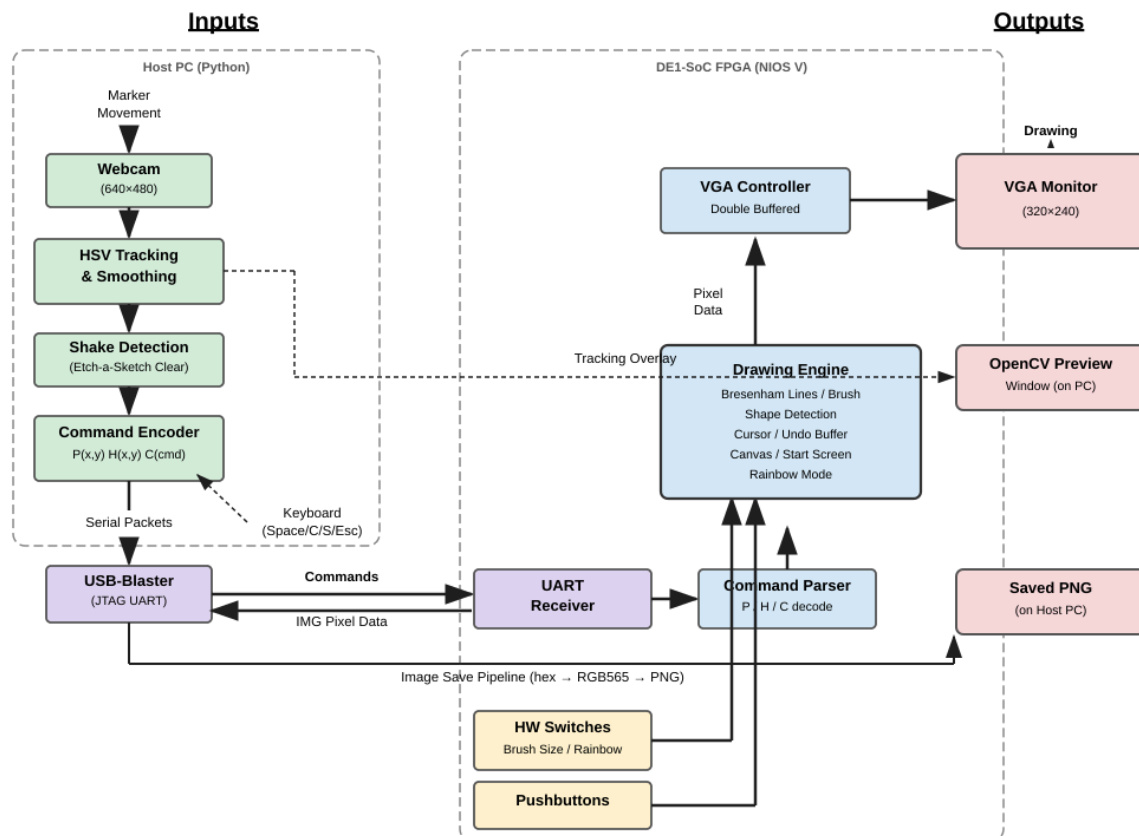
ECE243 Final Project Report

Thenuk Fernando & Luca Mammone

Project Description

Our project "PixelDraw" is an interactive drawing system that turns physical marker movements into real-time artwork displayed on a VGA monitor, utilizing the DE1-SoC FPGA board. A host PC's webcam runs a Python script using OpenCV to track a green marker using HSV color-space filtering. The (x,y) coordinates are detected and smoothed with an exponential moving average filter and transmitted over to a NIOS V processor over JTAG UART. The NIOS V program receives the coordinates and renders brush strokes onto a 320x240 VGA pixel buffer via Bresenham's line algorithm, allowing support for variable brush sizes and rainbow color mode (toggled by hardware switches). Furthermore, the system includes features such as shape detection logic classifying drawn strokes as lines, triangles, rectangles, or circles, utilizing vertex extraction and circularity metric. Additional features include etch-a-sketch shake-to-clear gesture (detected by Python script), an undo buffer, cursor preview, FPGA canvas saving to PC as a PNG image, and a decorative start screen.

System Block Diagram



How to Operate

Setup: Connect the DE1-SoC board to the VGA monitor and to the host PC via USB blaster. Ensure the webcam is connected to the PC. Program the FPGA with the compiled Nios V project and launch the Python tracking script on the host PC. Wait for the camera warm-up period (~3 seconds), after which the terminal prints "Ready."

Drawing: Hold a green marker in front of a webcam. Press spacebar to toggle between hovering and recording modes. In recording mode, moving the marker draws brush strokes on the VGA in real-time. The Python preview window will show the tracked contour and smoothed positions. After drawing press the spacebar again to lock in the brush strokes.

Controls: Press 'C' to clear the canvas. Shake the marker sideways rapidly to trigger an Etch-A-Sketch clear. Press 'S' to save the current canvas onto the PC as a PNG image. Use hardware switches on the DE1-SoC to alter modes and values, switches 7 (blue), 8 (green), 9 (red) control colour, 4, 5 and 6 control brush stroke thickness (1-8), switch 3 controls rainbow mode, switch 2 controls symmetry mode, and switches 1 and 0 control fill and shape detection mode. The pushbuttons also control the drawing and startup, key 0 is pressed to commit a previewed drawing to the canvas, key 1 is used as an undo key, key 2 is used as a start button, and key 3 restart. Finally press ESC to exit the application.

Work Attribution

The attribution table below summarizes the division of work between partners. Percentages reflect an approximate share of effort for each module.

Task	TF %	LM %	Thenuk Fernando	Luca Mammone
Python OpenCV Tracking Script	50%	50%	HSV calibration, contour detection, smoothing logic	JTAG UART communication, camera setup, shake detection
VGA Drawing)	50%	50%	VGA pixel buffer setup, double-buffering	Bresenham line drawing, brush rendering
Shape Detection Algorithm	40%	60%	Bounding box and area calculations	Vertex extraction, polygon classification, circularity metric
JTAG UART Protocol Design	45%	55%	Command encoding scheme (P/H/C prefixes)	UART read/write helpers, parser on NIOS side
Cursor & Undo System	55%	45%	Cursor save/restore logic, undo buffer implementation	Erase cursor rendering, VGA swap integration
Image Save (FPGA to PC)	35%	65%	Testing and validation of saved images	Hex pixel encoding, IMG_START/END protocol, PNG export
Start Screen & UI	60%	40%	Start screen assembly procedure, UI	Canvas ornamental border rendering
Integration & Testing	50%	50%	End-to-end testing, debugging	Performance tuning, threshold calibration